

Knowledge Distillation for Aerial Robots Control

Alessio Salvatore¹, Angelo Stefanini¹; Supervision: Dr. Eugenio Cuniato² and Dr. Thomas Stastny²

Abstract—Overactuated tilt-rotor aerial robots can generate forces independently of their attitude, enabling physical interaction and robust flight near surfaces. However, this actuation redundancy makes control challenging, especially when the real platform differs from its identified model or is affected by external disturbances. Existing model-based controllers can use disturbance observers to handle model mismatch and external forces, but observer bandwidth, measurement noise, and tuning can limit fast adaptation. We address this limitation by distilling multiple expert model-based controllers into a single recurrent neural controller. Each expert is matched to a different simulated plant, providing the student with control behaviors across a distribution of system parameters. By learning from this family of experts, the recurrent policy captures how the control action should adapt to different dynamics during flight. The resulting controller stabilizes the omnidirectional aerial robot, improves robustness to model mismatch, and transfers zero-shot to the real platform.

Index Terms—OMAVs, robust control, knowledge distillation, imitation learning, sim-to-real transfer.

I. INTRODUCTION

In recent years, omnidirectional micro aerial vehicles (OMAVs) have attracted increasing attention in aerial robotics. By independently tilting their propeller arms, these overactuated platforms can generate a full six-degree-of-freedom control wrench, decoupling translational motion from body attitude and enabling omnidirectional flight [1]. This capability makes OMAVs especially well suited for physical interaction with the environment, including inspection and maintenance tasks in complex or hard-to-access structures [2], [3].

However, this overactuation makes control more challenging than for conventional fixed-rotor vehicles. Tilt-rotor platforms require allocation methods that manage actuator redundancy, avoid singularities, and account for the different response times of tilt motors and propellers [4]. Existing OMAV controllers typically rely on a layered architecture, where a PID/PD or model-based controller computes a desired six-dimensional wrench that is then mapped to individual actuators through control allocation. More advanced solutions combine model-based control with disturbance observers to improve tracking and interaction performance under model mismatch and external forces [5]. However, these observers depend on bandwidth selection, noisy velocity or force measurements, and careful tuning. In practice, fast-changing dynamics or unmodeled interaction effects can therefore degrade tracking performance.



Fig. 1. An omnidirectional aerial vehicle in free flight.

These limitations have motivated growing interest in learning-based control for OMAVs. Rather than explicitly modeling all aerodynamic, contact, and actuator effects, learning-based methods shift part of this complexity offline by training policies in simulation under randomized dynamics and disturbances. Recent work has used reinforcement learning to generate reactive manipulation behaviors for aerial door opening, improving robustness to perception errors and task variations [6]. Similarly, end-to-end learned controllers have been proposed to directly coordinate tilt arms and propellers for OMAV flight, showing improved robustness to inertial and force disturbances compared to traditional controllers [7]. However, purely learned policies often require large amounts of simulated interaction and careful sim-to-real tuning, while offering fewer of the structural guarantees associated with model-based control.

Knowledge distillation offers a promising way to bridge these two directions. Instead of learning a controller from scratch, distillation transfers the behavior of one or multiple expert policies into a compact student model [8], [9]. In the context of OMAV control, this suggests a natural but largely unexplored alternative: using model-based controllers as structured experts, while training a recurrent neural policy to reproduce their behavior across different system dynamics. This creates a middle ground between model-based and learned control: the experts provide reliable, physically grounded control behaviors, while the recurrent student learns to adapt them to model mismatch and parameter variations.

¹MSc Robotics, Systems and Control, *ETH Zürich*

²Autonomous Systems Lab, *ETH Zürich*

A. Contributions

This work investigates whether model-based OMAV control behavior can be distilled into a recurrent neural policy that remains effective across different parameter configurations. We propose a multi-teacher distillation framework in which model-based controllers, each matched to a different simulated plant, supervise a single end-to-end Long-Short Term Memory controller. The student learns to reproduce the control behavior of these matched experts. Specifically, we propose the following contributions:

- We demonstrate that the distilled policy controls the OMAV in closed-loop simulation, outperforming the nominal model-based controller in both tracking accuracy and robustness to parameter variations while requiring substantially less simulated interaction than end-to-end reinforcement-learning approaches.
- We show that the recurrent policy exhibits emergent system-identification capabilities, implicitly inferring the plant dynamics from its interaction history and adapting its control behavior accordingly, and validate its zero-shot transfer to the real platform.

B. Related work

Knowledge distillation has recently been used to train general-purpose robot policies that retain specialized expert behaviors while generalizing across platforms. In legged locomotion, morphology-specific controllers have been distilled into a single morphology-agnostic policy, demonstrating that teacher–student learning can preserve robot-specific control strategies within a universal controller [10]. In aerial robotics, specialized quadrotor policies have been distilled into a recurrent student that adapts online to unseen dynamics through its interaction history, effectively performing implicit system identification [11]. Together, these examples motivate the use of multi-teacher distillation for robust control across varying plant dynamics.

II. MODEL AND CONTROL

We consider an overactuated omnidirectional aerial vehicle equipped with five independently tiltable propeller arms, as shown in Fig. 1. The actuator command at time t is defined

$$\mathbf{u}_t = \begin{bmatrix} \boldsymbol{\alpha}_t \\ \boldsymbol{\omega}_t \end{bmatrix} \in \mathbb{R}^{10}, \quad (1)$$

where $\boldsymbol{\alpha}_t, \boldsymbol{\omega}_t \in \mathbb{R}^5$ denote the arm-tilt and propeller-speed setpoints, respectively.

The vehicle velocity is represented by the body twist

$$\boldsymbol{\nu}_t = \begin{bmatrix} {}^B \mathbf{v}_t \\ {}^B \boldsymbol{\Omega}_t \end{bmatrix} \in \mathbb{R}^6, \quad (2)$$

where ${}^B \mathbf{v}_t$ and ${}^B \boldsymbol{\Omega}_t$ are the linear and angular velocities expressed in the body-fixed frame $\mathcal{F}_B$, whose origin is located at the vehicle center of mass. Given the reference $\boldsymbol{\nu}_t^{\text{ref}}$, the tracking error is defined as

$$\mathbf{e}_{\nu,t} = \boldsymbol{\nu}_t^{\text{ref}} - \boldsymbol{\nu}_t. \quad (3)$$

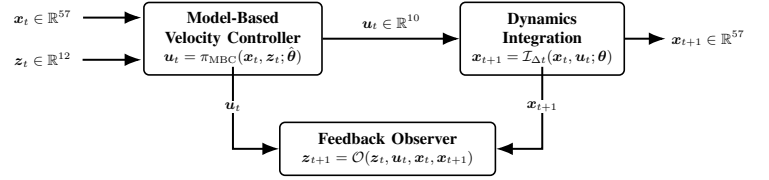


Fig. 2. High-level representation of the model-based velocity controller, plant dynamics, and recurrent observer update.

Let $\boldsymbol{\theta}$ denote the physical plant parameters and $\hat{\boldsymbol{\theta}}$ the corresponding values assumed by the controller. At each control step, the model-based controller computes

$$\mathbf{u}_t^{\text{MBC}} = \pi_{\text{MBC}}(\mathbf{x}_t, \mathbf{z}_t; \hat{\boldsymbol{\theta}}), \quad (4)$$

where $\mathbf{x}_t \in \mathbb{R}^{57}$ collects the velocity reference, measured velocities, tracking errors, and actuator feedback, while $\mathbf{z}_t \in \mathbb{R}^{12}$ is the feedback-observer state. This observer state encodes recent input–output discrepancies and provides an estimate of the disturbances and model errors to be compensated by the controller. The controller computes a desired body wrench from the tracking error and observer compensation, and maps it to $\mathbf{u}_t^{\text{MBC}}$ through the control-allocation layer.

After applying $\mathbf{u}_t^{\text{MBC}}$ over the sampling interval Δt , the plant evolves according to $\boldsymbol{\theta}$ and produces the next controller input $\mathbf{x}_{t+1}$. The observer then updates its state using $\mathbf{z}_t$, $\mathbf{u}_t^{\text{MBC}}$, $\mathbf{x}_t$, and $\mathbf{x}_{t+1}$, thereby retaining information about the recent input–output history. The resulting closed-loop architecture is shown in Fig. 2.

III. PROBLEM FORMULATION

Let $\boldsymbol{\theta}_i \in \mathcal{P}$ denote the physical-parameter vector of the i -th simulated OMAV configuration, sampled from the plant distribution $p(\boldsymbol{\theta})$. For each configuration, we consider a parameter-matched model-based teacher

$$\mathbf{u}_t^{(i)} = \pi_{\text{MBC}}(\mathbf{x}_t^{(i)}, \mathbf{z}_t^{(i)}; \boldsymbol{\theta}_i). \quad (5)$$

The superscript (i) identifies quantities associated with the i -th plant–teacher pair.

Our objective is to distill the behavior of multiple parameter-matched teachers into a single recurrent policy

$$(\hat{\mathbf{u}}_t, \mathbf{h}_{t+1}, \mathbf{c}_{t+1}) = \pi_{\phi}(\mathbf{x}_t, \mathbf{h}_t, \mathbf{c}_t), \quad (6)$$

where ϕ denotes the trainable parameters, while $\mathbf{h}_t$ and $\mathbf{c}_t$ are the LSTM hidden and cell states, respectively. Unlike the model-based teachers, the student does not receive $\boldsymbol{\theta}_i$ explicitly and must produce suitable actuator commands across different plant configurations from the observed input history.

The objective is therefore to preserve the behavior of parameter-matched controllers while improving robustness to plant–controller mismatch, which occurs when the physical parameters $\boldsymbol{\theta}$ differ from the parameters $\hat{\boldsymbol{\theta}}$ assumed by a nominal controller. An analysis of the effect of such mismatches on the nominal controller’s velocity-tracking performance is provided in the Appendix, Table III.

IV. METHODOLOGY

Our methodology comprises two stages. First, parameter-matched model-based controllers generate expert demonstrations across a distribution of simulated OMAV dynamics. These demonstrations are then used to train a single recurrent policy through closed-loop differentiable rollouts.

A. Expert Dataset Generation

1) *Plant Randomization and Teacher Pairing:* We simulate $N = 30000$ OMAV configurations. For each configuration i , the plant parameters θ_i are obtained by perturbing the nominal multibody-dynamics parameters to represent plausible variations in mass, inertia, geometry, and aerodynamic coefficients. Full details are reported in Appendix , Table I.

Each plant is paired with a model-based teacher using the corresponding parameters:

$$\mathbf{u}_t^{(i)} = \pi_{\text{MBC}}(\mathbf{x}_t^{(i)}, \mathbf{z}_t^{(i)}; \theta_i), \quad (7)$$

where $\mathbf{u}_t^{(i)}$ denotes the actuator command generated by the i -th teacher. Since the teacher and plant share the same parameters θ_i , the demonstrations are generated without plant-controller parameter mismatch.

2) *Reference Generation and Rollouts:* Each rollout tracks a constant body-twist reference ν^{ref} , sampled from the distribution reported in Appendix , Table II. This distribution includes isolated and coupled linear and angular velocity commands, ensuring excitation across the controlled degrees of freedom under representative operating conditions.

Each rollout is initialized with randomized body-frame linear and angular velocities and an identity attitude. The matched teacher controls the corresponding plant 5 s rollout at 200 Hz.

At each step, we record the teacher input $\mathbf{x}_t^{(i)} \in \mathbb{R}^{57}$, actuator command $\mathbf{u}_t^{(i)} \in \mathbb{R}^{10}$, and tracking error $\mathbf{e}_{\nu,t+1}^{(i)} \in \mathbb{R}^6$, as visualized in Fig. 3. The corresponding plant parameters θ_i are also retained for dynamics integration during training. The resulting dataset contains approximately 42 h of simulated flight.

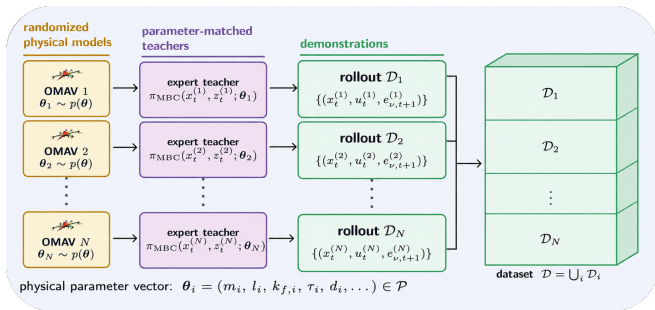


Fig. 3. From left to right, randomized physical models are sampled, matched with expert teachers and rolled out to generate demonstrations. The resulting trajectories are combined into a single dataset covering the full range of model variations.

B. Recurrent Policy Training

We choose a Long Short-Term Memory (LSTM) network [12] as the student policy, as its gated recurrent architecture is well suited to processing temporally correlated observations and actions in closed loop. A comparison with alternative student architectures is provided in Sec. V-C.

For each training window, the student is paired with a specific plant and the corresponding parameter-matched teacher. The recurrent student policy is

$$(\hat{\mathbf{u}}_t, \mathbf{h}_{t+1}, \mathbf{c}_{t+1}) = \pi_\phi(\hat{\mathbf{x}}_t, \mathbf{h}_t, \mathbf{c}_t), \quad (8)$$

where ϕ denotes the trainable parameters, while $\mathbf{h}_t$ and $\mathbf{c}_t$ are the LSTM hidden and cell states. The same policy parameters ϕ are shared across all plant configurations.

Each window is initialized from a teacher state, $\hat{\mathbf{x}}_0 = \mathbf{x}_0^{(i)}$. At each step, the student command is applied to the differentiable dynamics of the selected plant:

$$\hat{\mathbf{x}}_{t+1} = \mathcal{I}_{\Delta t}(\hat{\mathbf{x}}_t, \hat{\mathbf{u}}_t; \theta_i). \quad (9)$$

The resulting state becomes the next student input. Thus, within each window, the student learns from the teacher matched to one specific plant. Across training windows, it is exposed to all plant-teacher pairs.

The training objective combines command imitation and closed-loop behavior matching:

$$\begin{aligned} \mathcal{L}(\phi) &= \frac{1}{T} \sum_{t=0}^{T-1} \mathcal{L}_t \\ &= \frac{1}{T} \sum_{t=0}^{T-1} \left[\lambda_u \left\| \hat{\mathbf{u}}_t - \mathbf{u}_t^{(i)} \right\|_2^2 + \lambda_y \left\| \hat{\mathbf{e}}_{\nu,t+1} - \mathbf{e}_{\nu,t+1}^{(i)} \right\|_2^2 \right]. \end{aligned} \quad (10)$$

For the selected plant, the first term encourages the student to reproduce the matched teacher commands, while the second matches its velocity-tracking behavior. Repeating this procedure across all plants distills the specialized teachers into a single recurrent policy. The policy and plant dynamics

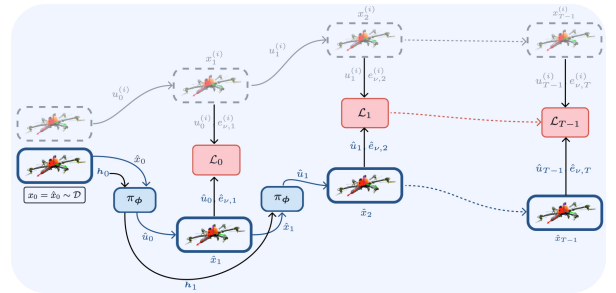


Fig. 4. Training begins from an initial state sampled from the demonstration dataset. The student policy then rolls out autonomously, generating actions and future states over the prediction horizon. Behavioral cloning losses are computed against the corresponding demonstration trajectory and accumulated across the rollout

are jointly unrolled for T steps as visualized in Fig. 4. Back-propagation through time propagates the loss through the LSTM memory and the differentiable dynamics [13], [14],

accounting for both the immediate and accumulated effects of the student commands.

C. Training Setup

The student consists of three long short-term memory layers with hidden dimension 128. Training uses $T = 50$ steps per window, a batch size of 2048, and 20 epochs, with $\lambda_u = 1/6$ and $\lambda_y = 10$. Gaussian input noise with standard deviation $\sigma_n = 0.01$ is applied during training. Optimization uses Adam, cosine learning-rate annealing from 10^{-6} to 10^{-7} , and gradient-norm clipping at 1. Full training requires approximately one day on an NVIDIA RTX 4000 GPU.

V. RESULTS

We first evaluate the distilled controller in closed-loop simulation across different plant configurations and parameter mismatches. We then deploy the same policy on the real OMAV without additional fine-tuning.

A. Closed-Loop Evaluation

We evaluate in simulation the controllers on a test set of 2,000 paired closed-loop rollouts, each lasting 5 s. The test set covers perturbed OMAV configurations with plant parameters θ_i and different velocity references. For each plant, the distilled recurrent policy π_ϕ is compared against the nominal model-based controller, parameterized by $\hat{\theta}$, and a matched model-based controller whose parameters coincide with those of the plant, i.e., θ_i .

For a rollout of length H , tracking performance is quantified by the mean linear and angular velocity errors

$$\bar{e}_v = \frac{1}{H} \sum_{t=0}^{H-1} \|B \mathbf{v}_t^{\text{ref}} - B \mathbf{v}_t\|_2, \quad \bar{e}_\Omega = \frac{1}{H} \sum_{t=0}^{H-1} \|B \Omega_t^{\text{ref}} - B \Omega_t\|_2 \quad (11)$$

These metrics measure the average tracking error over the complete rollout. Fig. 5 shows their distributions across the test set, grouped into bins according to the magnitude of the corresponding reference velocity.

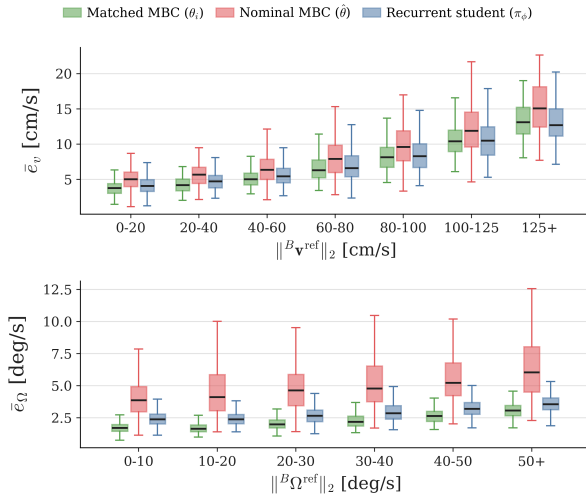


Fig. 5. Closed-loop velocity-tracking performance of the nominal MBC, matched teacher, and recurrent student.

The recurrent policy consistently reduces tracking errors compared with the nominal MBC across the reference-velocity range, while approaching the performance of the parameter-matched MBC. The improvement is statistically significant for both linear and angular tracking over the 2,000 paired rollouts, as confirmed by paired Wilcoxon signed-rank tests with Holm correction ($p < 10^{-10}$).

We observe that the student network improves tracking performance particularly in the angular components. We interpret this as a consequence of the stronger coupling and higher sensitivity of the rotational dynamics to variations in the plant parameters.

Robustness is further assessed under progressively increasing plant-parameter randomization around the nominal configuration. As the mismatch between the randomized plant parameters θ_i and the fixed parameters $\hat{\theta}$ assumed by the nominal controller increases, the recurrent student consistently exhibits a lower divergence rate than the nominal model-based controller. A rollout is considered divergent if it fails to reduce the tracking error below a fixed threshold within the rollout window.

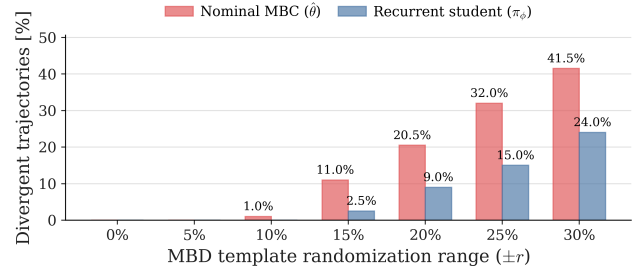


Fig. 6. Out-of-distribution robustness under increasing plant-parameter mismatch.

B. Emergent System Identification

To examine the information encoded in the recurrent state, we train a linear probe to predict the plant thrust-to-weight ratio from $\mathbf{h}_t$, following RAPTOR [11]. As shown in Fig. 7, the probe achieves an R^2 of 0.967 on held-out data and rapidly converges to the correct value during a rollout. Since the plant parameters are never observed by the policy, the accurate linear decoding of this latent physical quantity suggests that the recurrent controller implicitly identifies the controlled plant and encodes this information in its hidden state.

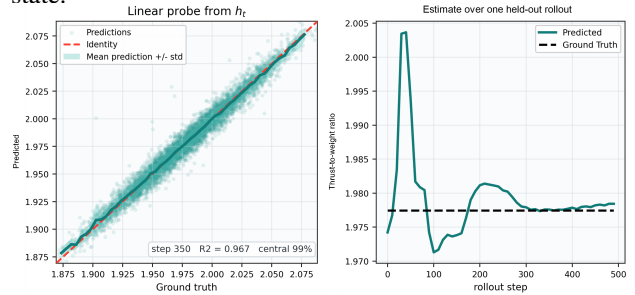


Fig. 7. Linear probing of $\mathbf{h}_t$ for the plant thrust-to-weight ratio. Left: held-out predictions with binned mean and standard deviation; right: probe estimate over one rollout versus ground truth.

C. Architecture Comparison

We compare the LSTM student with a memoryless MLP, an MLP receiving an explicit history of the ten most recent observations, and a GRU-based recurrent policy. All architectures are trained and evaluated under identical conditions using the same dataset, objective, and rollout settings. Performance is assessed using the mean linear and angular tracking errors computed over the complete rollout.

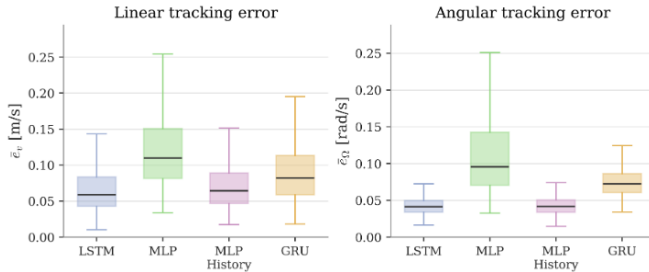


Fig. 8. Distributions of the mean linear and angular tracking errors over complete rollouts for the evaluated student architectures.

As shown in Fig. 8, the LSTM achieves the best overall tracking performance, particularly compared with the memoryless MLP. The improvement obtained by architectures with access to temporal information indicates that memory is important for this control problem. Since the plant parameters are not directly observed, the instantaneous policy input does not fully characterize the underlying dynamics. Recurrent states or explicit observation histories can therefore reduce this partial observability by retaining information from previous plant responses and control actions.

D. Zero-Shot Transfer to the Real Platform

The trained recurrent policy is deployed on the real OMAV without fine-tuning. Despite unmodeled delays, actuator imperfections, and the large sim-to-real gap of an overactuated aerial platform, the lightweight controller runs on constrained onboard hardware while maintaining stable closed-loop velocity tracking and performing complex SE(3) maneuvers. This provides preliminary evidence that the distilled policy can transfer to the real system beyond the simulated plant distribution.

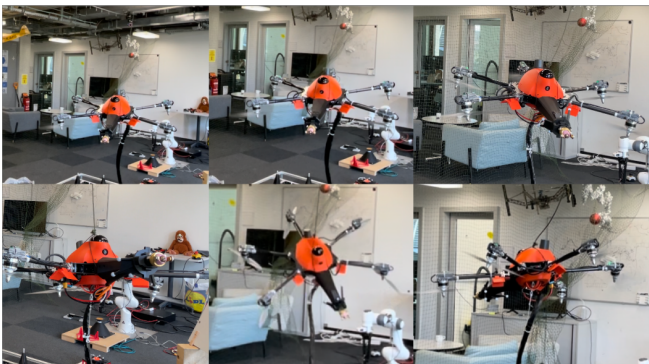


Fig. 9. Zero-shot deployment of the recurrent controller on the real OMAV.

VI. CONCLUSION

This work presented a multi-teacher knowledge-distillation framework for robust control of an omnidirectional aerial vehicle. A recurrent student was trained from model-based controllers matched to randomized plant configurations and subsequently deployed without direct access to the plant parameters. In simulation, the distilled policy improved tracking accuracy and robustness to model mismatch compared with the nominal model-based controller, while approaching the performance of parameter-matched teachers. The resulting controller combines the adaptability and computational efficiency of a neural policy with the physically grounded control behavior provided by the model-based teachers. Linear probing further showed that the recurrent state encodes latent physical properties of the controlled plant, providing evidence of implicit system identification. Despite being trained only in simulation, the lightweight policy generalizes zero-shot to the real OMAV, where it maintains stable closed-loop velocity tracking and performs complex SE(3) maneuvers.

Importantly, our approach is substantially less data-hungry than end-to-end reinforcement-learning methods for OMAV control: while prior work reports 87 days of simulated interaction [7], our student is trained with less than 2 days of simulated flight data.

Although preliminary real-platform experiments are promising, extensive validation on the physical OMAV remains necessary to assess robustness under unmodeled aerodynamic effects, sensor noise, delays, and hardware variability. Future work will therefore focus on systematic real-world evaluation and adaptation. The distilled controller could also provide a structured initialization for reinforcement learning, enabling subsequent task-specific optimization while retaining the stable control behavior learned from the model-based teachers.

REFERENCES

- [1] A. Ollero, M. Tognon, A. Suarez, D. Lee, and A. Franchi, “Past, present, and future of aerial robotic manipulators,” *IEEE Transactions on Robotics*, vol. 38, no. 1, pp. 626–645, Feb. 2022.
- [2] A. Ollero, G. Heredia, A. Franchi, G. Antonelli, K. Kondak, A. Sanfeliu, A. Viguria, J. R. Martinez-De Dios, F. Pierri, J. Cortes, A. Santamaria-Navarro, M. A. T. Soto, R. Balachandran, J. Andrade-Cetto, and A. Rodriguez, “The AEROARMS project: Aerial robots with advanced manipulation capabilities for inspection and maintenance,” *IEEE Robotics Automation Magazine*, vol. 25, no. 4, pp. 12–23, Dec. 2018.
- [3] B. Chan, H. Guan, J. Jo, and M. Blumenstein, “Towards UAV-based bridge inspection systems: A review and an application perspective,” *Structural Monitoring and Maintenance*, vol. 2, no. 3, pp. 283–300, 2015.
- [4] E. Cuniato, M. Allenspach, T. Stastny, H. Oleynikova, R. Siegwart, and M. Pantic, “Allocation for omnidirectional aerial robots: Incorporating power dynamics,” *IEEE Transactions on Robotics*, early access, 2026, Art. no. 11498628.
- [5] K. Bodie, M. Brunner, M. Pantic, S. Walser, P. Pfandler, U. Angst, R. Siegwart, and J. Nieto, “Active interaction force control for contact-based inspection with a fully actuated aerial vehicle,” *IEEE Transactions on Robotics*, vol. 37, no. 3, pp. 709–722, Jun. 2021.
- [6] E. Cuniato, I. Geles, W. Zhang, O. Andersson, M. Tognon, and R. Siegwart, “Learning to open doors with an aerial manipulator,” *arXiv preprint arXiv:2307.15581*, Jul. 2023.

- [7] E. Cuniato, O. Andersson, H. Oleynikova, R. Siegwart, and M. Pantic, "Learning to fly omnidirectional micro aerial vehicles with an end-to-end control network," *arXiv preprint arXiv:2312.05125*, Dec. 2023.
- [8] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, Mar. 2015.
- [9] A. A. Rusu, S. G. Colmenarejo, C. Gulcehre, G. Desjardins, J. Kirkpatrick, R. Pascanu, V. Mnih, K. Kavukcuoglu, and R. Hadsell, "Policy distillation," *arXiv preprint arXiv:1511.06295*, Nov. 2015.
- [10] W. Xi, Z. Cao, C. Ming, J. Zheng, and G. Zhou, "UniLegs: Universal multi-legged robot control through morphology-agnostic policy distillation," *arXiv preprint arXiv:2507.22653*, 2025.
- [11] J. Eschmann, D. Albani, and G. Loianno, "RAPTOR: A foundation policy for quadrotor control," *arXiv preprint arXiv:2509.11481*, 2026.
- [12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [13] P. J. Werbos, "Backpropagation through time: What it does and how to do it," *Proceedings of the IEEE*, vol. 78, no. 10, pp. 1550–1560, 1990.
- [14] F. de Avila Belbute-Peres, K. Smith, K. Allen, J. Tenenbaum, and J. Z. Kolter, "End-to-end differentiable physics for learning and control," in *Advances in Neural Information Processing Systems*, 2018, pp. 7178–7189.

APPENDIX

This appendix summarizes the plant-parameter randomization, reference-velocity distribution, and sensitivity analysis used in this work. Position and inertia quantities are expressed in the frame indicated by their left superscript: $\mathcal{F}_B$ for the body, $\mathcal{F}_A$ for the arms, and $\mathcal{F}_R$ for the rotors.

TABLE I
PLANT-PARAMETER RANDOMIZATION USED FOR EXPERT-DATA
GENERATION.

Parameter	Symbol	Randomization range
Reference-frame origin	${}^B\mathbf{p}_{B,O}$	additive ± 0.01 m per axis
Base mass	m_{base}	additive ± 0.10 kg
Base center of mass	${}^B\mathbf{p}_{B,BM}$	additive ± 0.01 m per axis
Base inertia	${}^B\mathbf{I}_{BM}$	multiplicative $\pm 10\%$
Arm masses	$m_{\text{arm},i}$	multiplicative $\pm 10\%$
Arm centers of mass	${}^A\mathbf{p}_{A,AM,i}$	additive ± 0.01 m and multiplicative $\pm 10\%$
Arm inertia	${}^A\mathbf{I}_{AM,i}$	multiplicative $\pm 10\%$; zero terms additive ± 0.001 kg m ²
Arm-joint origins	${}^B\mathbf{p}_{B,AO,i}$	additive ± 0.01 m; selected terms also multiplicative $\pm 10\%$
Tilt-angle offsets	$\Delta\alpha_i$	additive ± 0.01 rad
Rotor masses	$m_{\text{rotor},i}$	multiplicative $\pm 10\%$
Rotor centers of mass	${}^R\mathbf{p}_{R,RM,i}$	z component: additive ± 0.01 m and multiplicative $\pm 10\%$
Rotor inertia	${}^R\mathbf{I}_{RM,i}$	multiplicative $\pm 10\%$
Rotor-joint origins	${}^A\mathbf{p}_{A,RO,i}$	additive ± 0.01 m; selected terms also multiplicative $\pm 10\%$
Propeller origins	${}^R\mathbf{p}_{R,PO,i}$	z component: additive ± 0.01 m
Static-thrust coefficients	$k_{f,i}$	multiplicative $\pm 5\%$
Drag-torque coefficients	$k_{m,i}$	multiplicative $\pm 5\%$

All velocity references are expressed in $\mathcal{F}_B$. The reference modes include both isolated and coupled linear and angular commands. "All axes" denotes simultaneous commands along the three body-frame axes, whereas "one axis" denotes a randomly selected axis. Yaw refers to angular velocity about the body z -axis.

TABLE II
BODY-FRAME REFERENCE-VELOCITY DISTRIBUTION USED FOR
EXPERT-DATA GENERATION.

Reference mode	Fraction	Linear velocity	Angular velocity
Random	19 %	all axes	all axes
Single linear + yaw	20 %	one axis	yaw
Full linear + yaw	24 %	all axes	yaw
Full linear	15 %	all axes	—
Yaw only	7 %	—	yaw
Single angular	3 %	—	one axis
Single linear	12 %	one axis	—

A. Sensitivity Analysis

To determine which physical parameters most strongly affect closed-loop tracking performance, we perturb each of the $p = 12$ multibody-dynamics parameter groups by up to $\pm 20\%$ of its nominal value. The nominal controller is then evaluated on a quasi-Monte Carlo ensemble of $n = 16,359$ perturbed plants.

Performance is quantified using the time-weighted integral of the normalized linear and angular tracking errors:

$$J = \int_0^{5\text{ s}} t \left(\frac{\|\mathbf{v}_t - \mathbf{v}_t^{\text{ref}}\|_2}{s_{\text{lin}}} + w \frac{\|\boldsymbol{\Omega}_t - \boldsymbol{\Omega}_t^{\text{ref}}\|_2}{s_{\text{ang}}} \right) dt, \quad w = 1, \quad (12)$$

where s_{lin} and s_{ang} are the linear and angular normalization factors, respectively. Parameter sensitivities are estimated as the standardized ordinary-least-squares coefficients $\hat{\beta}_i$ obtained by regressing the standardized cost on the standardized perturbation factors.

TABLE III
STANDARDIZED SENSITIVITY OF THE TRACKING COST J TO
MULTIBODY-DYNAMICS PARAMETER PERTURBATIONS OF UP TO $\pm 20\%$.

Parameter	$\hat{\beta}$	t -statistic	Variance share
Static-thrust coefficient	−0.1793	−23.438	0.7421
Base mass	0.0957	12.505	0.2113
Arm masses	0.0370	4.837	0.0316
Rotor masses	0.0159	2.073	0.0058
Arm-joint origins	0.0102	1.336	0.0024
Rotor inertia	−0.0086	−1.118	0.0017
Drag-torque coefficient	−0.0082	−1.078	0.0016
Arm inertia	−0.0070	−0.921	0.0011
Arm centers of mass	−0.0068	−0.884	0.0011
Base inertia	−0.0047	−0.612	0.0005
Rotor centers of mass	−0.0045	−0.584	0.0005
Base center of mass	−0.0031	−0.404	0.0002

The standardized regression coefficients in Table III indicate that the tracking cost is primarily influenced by the static-thrust coefficient and the base mass, which account for approximately 74% and 21% of the attributed variance, respectively. All remaining parameter groups jointly account for less than 5%.